DREAMWORLD

The 3D Internet

P R O D U C T R E Q U I R E M E N T S D O C U M E N T

Maximalist Vision | v1.0

roancurtis.com/dreamworld

Roan Curtis | February 2026

1. Executive Summary

Dreamworld is a 3D spatial internet platform where users inhabit persistent worlds, interact with AI agents, build anything through natural language, and connect through a universal graph-based multiverse. It replaces the flat, document-based internet with a spatial, physics-enabled, AI-native environment where every surface can be a screen, every file is a physical object, and every program runs on real infrastructure accessible from within the 3D world.

The platform is designed as a living organism with interconnected systems that communicate through a universal Packet protocol. It supports sandbox creation, multiplayer survival gameplay, AI-assisted development, local-first privacy, and an open-source engine monetized through hosted services.

Core URL
roancurtis.com/dreamworld

Component	Specification
Platform	Browser-based (Three.js/WebGPU), Mobile (Telegram bot), Desktop (future)
Backend	Node.js gateway + world processes, Python agent/knowledge services
Database	PostgreSQL with spatial indexing
Multiplayer	WebSocket (cloud), WebSocket (LAN), WebRTC (future P2P)
AI Integration	Multi-agent hierarchical system with personal knowledge graphs
Monetization	Token economy, marketplace fees, enterprise licensing
License	Open-source engine, proprietary hosted service
Target Launch	3-month polished MVP (single-player house + computer + agent)

2. Vision and Philosophy

2.1 What Dreamworld Is

Dreamworld is the 3D internet. Websites are worlds. Hyperlinks are portals. URLs are node IDs. A browser is the in-game computer. An iframe is a screen running someone else's site inside your world. HTTP requests are Packets. A search engine is the Knowledge Graph query interface. The current internet is a 2D document system built on hyperlinks between pages. Dreamworld is the same thing but the documents are worlds, the hyperlinks are portals with actual spatial position, the browser is your body walking through 3D space, and other people are visible and present everywhere.

2.2 Design Principles

- **Soul over features.** Every component must share a consistent visual language, animation style, and interaction philosophy. Warm minimalism with depth. Nothing ships until it feels right.
- **Local-first privacy.** Personal data lives on the user's device. The server processes intelligence ephemerally but stores nothing private. Users control what crosses each privacy boundary.
- **Packets all the way down.** Every interaction, state change, agent thought, physics tick, and camera frame travels as a universal self-describing Packet. One format. One language. One protocol.
- **Open engine, proprietary ecosystem.** The code is open source. The hosted instance with its accumulated Knowledge Graph, community, marketplace, and agent infrastructure is the product.
- **Adding to the ecosystem is free. Extracting from it costs.** Creation, building, sharing, learning: all free. Commerce, business storefronts, enterprise deployments: paid.

2.3 The Organism Metaphor

Dreamworld is architecturally organized as a living organism with 16 interconnected organ systems. Each organ has a specific function and communicates with all others through the universal Packet system. This metaphor is not just conceptual; it directly maps to the codebase structure, deployment architecture, and development roadmap.

3. The Universal Packet Protocol

The Packet is the single unit of communication in the entire organism. Every signal, every state change, every frame rendered, every agent thought, every object created, every camera recording travels as a Packet. A Packet has two parts: a Header (self-describing metadata, same shape every time) and a Body (variable content described by the Header's schema field).

3.1 Packet Header Specification

Component	Specification
id	UUID v4 - unique identifier for every packet
type	Enum: FRAME, EVENT, OBJECT, AGENT_TASK, AGENT_OUTPUT, KNOWLEDGE, CAMERA_FILM, WORLD_STATE, PHYSICS_TICK, CHAT, AUTH, MODERATION
schema	String describing the body's structure so any receiver can parse it
origin	ID of creator: user_id, agent_id, system_id, or world_id
destination	Target: specific ID, 'broadcast', 'world:xyz', 'agent_cluster:abc'
timestamp	ISO 8601 from the Watch system (UTC internally)

world_id	Which world this packet belongs to (null for system-level packets)
parent_id	ID of parent packet if this is a response (null otherwise)
ttl	Time-to-live in seconds before expiry
priority	0-10 urgency scale (0 = background, 10 = critical)
permissions	Access control: public, private, group:xyz
checksum	Integrity verification hash

3.2 Packet Types and Schemas

- **FRAME:** Rendered game state for a single tick. Schema contains visible objects, lighting, camera transform.
- **EVENT:** Any interaction. Schemas: player_moved, player_interacted, trigger_fired, chat_message, report, token_transfer, drone_input, route_plan.
- **OBJECT:** Asset creation/modification. Schemas: object_config, avatar_update, audio_asset, texture_asset, physics_profile.
- **AGENT_TASK:** Work order for the AI system. Contains task description, constraints, context nodes from personal graph.
- **AGENT_OUTPUT:** Completed work from an agent. Contains generated config files, code, decisions, knowledge extractions.
- **KNOWLEDGE:** Knowledge Graph operations. Schemas: query, result, node_create, node_update, solution, request.
- **CAMERA_FILM:** Recording data. Schemas: recording_header, camera_transform, world_snapshot.
- **WORLD_STATE:** World structure changes. Schemas: snapshot, delta, fork, lod_preview.
- **PHYSICS_TICK:** Physics computation results. Contains object deltas (position, velocity, rotation, collisions).
- **CHAT:** Player-to-player or player-to-agent communication.
- **AUTH:** Authentication and session management.
- **MODERATION:** Content moderation actions and reports.

3.3 Wire Format

Phase 1: JSON over WebSocket for ease of debugging and development. Phase 2: MessagePack binary encoding for production efficiency. The Packet spec must be a versioned contract document that both client and server implementations conform to. All Packet validation code is shared between client and server (Node.js on both sides).

4. The World System (Skeleton + Skin)

4.1 Graph-Based Multiverse

The world is a graph. Each node is a world. Edges connect worlds. The graph is fractal: worlds contain subworlds contain subworlds infinitely deep. Nodes can be connected (visible in the constellation, reachable by traversal) or disconnected (private dimensions, accessible only via portal objects). The graph edge weight determines physical distance between nodes; nodes close enough together can merge into continuous terrain forming landmasses, continents, or archipelagos.

4.2 Default World Theme: Floating Cloud Islands

The flagship Dreamworld instance uses a sky-based aesthetic. Nodes are floating islands sitting on top of clouds. The space between islands is open sky. Players can fly between islands using planes, drones, or flight abilities. Communication between distant islands uses radio waves and infrared signals with physics-based propagation. The sky environment supports dynamic weather: fog, storms, clear skies, sunset cycles, all driven by the Watch system and affecting gameplay (fog blocks infrared signals, storms affect flight, etc).

4.3 World Variety Through Subgraphs

While the top-level constellation is floating sky islands, any subgraph can be anything. A portal on a cloud island can lead to an underwater civilization subgraph, a medieval fantasy kingdom, a cyberpunk cityscape, a space station network, or an abstract mathematical space. World configs define terrain type, gravity rules, physics parameters, lighting, atmosphere, and visual style independently per subgraph. This means the graph supports infinite aesthetic variety while maintaining a cohesive top-level identity.

4.4 Template Worlds and the Starting House

Every new user receives a template world: a floating island with a house. The house contains an office (desk, bookshelf, monitors), a living room, and a computer station. The house is a config file that users can fully customize: move furniture, add rooms, paint walls, change materials, expand the building, or demolish it entirely and start from scratch. Templates provide a functional starting point so no user spawns into an empty void.

4.5 Infinite Object Creation

Any object can be created through natural language description routed to the agent system. The agent decomposes the request, generates config files (object DNA), assigns physics profiles, creates interaction scripts, and optionally nests subworlds inside objects. Every object in the world has:

- **Config file (DNA):** JSON defining geometry, materials, scale, metadata.
- **Physics profile:** Mass, friction, restitution, drag, gravity scale, collision shape, constraints, custom force generators, triggers.
- **Interaction script:** Behaviors for touch, look, use, throw, combine, enter.
- **Optional subworld:** Walk inside any object and it becomes its own world.
- **Knowledge node:** Creator ID, timestamp, ratings, semantic tags, lineage.

4.6 Custom Physics Per World

- **Spherical gravity:** Walk on all sides of a cube, gravity pulls toward world center.
- **Directional gravity:** Standard gravity but configurable direction (upside-down worlds, sideways worlds).
- **Zero gravity:** Space drift, momentum-only movement.
- **Variable zones:** Different gravity in different areas of the same world.
- **Non-Euclidean geometry:** Rooms longer inside than outside, self-connecting corridors, Escher-like spaces.
- **Custom weather:** Rain, fog, storms, clear skies. Affects physics (wind pushes objects, rain makes surfaces slippery) and signal propagation.
- **Time dilation:** Worlds can run at different clock speeds. 1 real minute = 1 game hour, or eternal twilight.

5. Camera, Character, and Movement

5.1 The Camera Spine

The camera system is the most important interaction mechanic. It uses a continuous zoom along a projected line from the character's head position backward and upward. Scroll wheel controls zoom level seamlessly across four zones:

Component	Specification
INSIDE (0-12%)	Camera at character center. Character mesh fades transparent. First-person interior view.
FIRST PERSON (12-30%)	Just outside character surface looking outward. Standard FPS perspective.
THIRD PERSON (30-60%)	Behind and above. Roblox Rival-style camera: orbitable, follows character with smooth interpolation, collision with walls prevents camera clipping inside geometry.
BIRD VIEW (60-100%)	Far back. Whole island visible, then constellation. Flying mode activated.

The third-person camera must match the polish level of Roblox/Rival: right-click drag to orbit, camera collision with environment so it never clips through walls, smooth follow with slight lag for cinematic feel, auto-centering when moving forward, instant snap to behind-character when entering combat.

5.2 Character System

5.2.1 Character Controller

Kinematic character controller (not physics-based) for responsive movement. Features: walking, running (hold shift), jumping, falling with gravity, climbing (specific surfaces), swimming (water zones), flying (bird view or vehicle). Slope handling: walk up gentle slopes, slide down steep ones.

Step-up: automatically step over small obstacles without jumping. Wall collision: slide along walls, don't stop dead.

5.2.2 Character Customization

Characters are built from templates that the creator (Roan) designs. Players select from available base models and customize:

- **Body:** Height, build, proportions from preset ranges.
- **Face:** Preset face types with color/feature adjustments.
- **Clothing:** Layered outfit system. Shirts, pants, jackets, hats, shoes, accessories. Mix and match from template library.
- **Tattoos:** Decal system. Select from tattoo library. Place on body parts. Scale, rotate, and position. Multiple tattoos can layer.
- **Colors:** Skin tone, hair color, eye color, clothing colors all adjustable.
- **Animations:** Walk, run, idle, jump, sit, wave, dance. Consistent across all character templates.

Character appearance data is stored as a compact config (~1-5KB) sent once when a player enters visual range, then cached. Appearance updates only transmit on change.

5.3 Flying and Vehicles

5.3.1 Personal Flight

In bird view, the character transitions to flying mode. Direct WASD + mouse control with physics-based drag and momentum. Fly between islands, explore the constellation, follow routes set on the map.

5.3.2 Planes

Craftable/purchasable vehicle objects with cockpit interiors (subworld inside the plane). Physics-based flight model: thrust, lift, drag, gravity. Fly to other islands. Can carry passengers. Plane has a dashboard with instruments: altimeter, speedometer, fuel gauge, compass, radio.

5.3.3 Drones

Remote-controlled camera platforms. Launch from your island, fly with separate controls while your character stays put. Camera view switches to drone perspective. Physics-based flight with limited battery/range. Uses: scouting, reconnaissance, photography, surveillance, racing. Destructible in PvP zones. Can carry small payloads.

5.3.4 Cars

Ground vehicles for large island surfaces. Physics-based: suspension, tire friction, engine torque, braking. Different vehicle types: fast cars, trucks (carry cargo), off-road vehicles. Craftable or purchasable.

5.3.5 Route Planning

From the holographic map table or the computer, tap a destination node. The system calculates a path through the graph (shortest, safest, or scenic). The route highlights as a glowing line through

the constellation. Options: teleport (instant, costs tokens), fly manually (follow the route), autopilot (vehicle follows route automatically, player can look around or go AFK), fast-travel subway system (automated transport network between major nodes).

6. The In-Game Computer and Screen System

6.1 Screen Object Architecture

Any surface in the game can become a screen. A screen is a PlaneGeometry mesh with an HTML canvas texture. When a player interacts with a screen, an HTML DOM overlay activates on top of the WebGL canvas, providing full web UI capabilities: React components, CSS layout, scroll, input fields, iframes.

The input bridge: raycasting hit on the 3D plane returns UV coordinates, multiplied by canvas resolution to get 2D pixel coordinates, then hit-tested against the UI element layout. This enables clicking buttons on screens in 3D space.

6.2 Monitor and Computer Objects

A Monitor is a screen object with a mount/stand. A Computer is a processing object with specs that correlate to a real VPS sub-container on the backend. Monitors connect to Computers via Wire objects (visible cables in the 3D world). A Computer's specs determine what it can run:

Component	Specification
Basic Computer	1 vCPU, 512MB RAM, 5GB storage. Simple apps, text editing, basic agent chat.
Standard Computer	2 vCPU, 2GB RAM, 20GB storage. Code editing, VSCode, moderate agent tasks.
Power Computer	4 vCPU, 8GB RAM, 50GB storage. Full development environment, multiple agent processes, heavy computation.
Supercomputer	8+ vCPU, 32GB RAM, 200GB storage. Enterprise workloads, multi-agent orchestration, ML training.

Users can upgrade their in-game computer, which corresponds to upgrading their container allocation on the backend. Higher-tier computers cost more tokens per month. When a computer is running, its VPS container is active; when powered off, the container pauses and costs nothing.

6.3 Wire System

Wires are visible connection objects between Monitors and Computers. They follow physics (drape, swing, can be tripped over) but are functional: they establish the data link between a display surface and a processing unit. Multiple monitors can connect to one computer. Wireless connections are a future upgrade item. Wires are cosmetic but functional: cutting a wire disconnects the monitor.

6.4 The Computer Operating System

When activated, the computer displays an OS interface as an HTML overlay. The OS is a state machine with apps:

- **Home/Hub:** Inspired by the Temporal Flow Oracle circular interface. Central node with app icons radiating outward.
- **Profile and Avatar:** Character customization, stats, friends list, reputation score.
- **Calendar:** Real-world time (UTC, user timezone) merged with game time events. Deadlines, greenhouse schedules, agent tasks, community events.
- **Map:** 3D holographic constellation graph. Set routes, see active worlds, locate friends, identify territory.
- **Agent Chat:** Talk to your AI agent team. Braindump interface. See task decomposition in real time.
- **Knowledge Graph Browser:** Navigate your personal subgraph and the shared knowledge layer in 3D.
- **File Manager / Bookshelf:** Physical bookshelf in the office. Each book = a file/knowledge node. Pull books off shelf to view contents.
- **Code Editor:** Either iframed VSCode (code-server) or custom agentic coding interface.
- **App Store:** Install community-created apps on your computer.
- **Settings:** World config, privacy controls, notification preferences, computer upgrade options.

6.5 Running External Programs on Screens

Any web application with a URL can be iframed onto any screen in the 3D world. A user's existing web apps running on their server can be displayed by pointing a screen at the URL. The postMessage API bridges the iframe to the game engine for bidirectional communication: the web app can read game state and emit Packets through the game engine.

6.6 Wall of Monitors

A room can contain multiple screen objects arranged as a panoramic workspace. Each screen runs a different app independently. The player sits in the center and focuses screens individually or views all simultaneously. One screen shows the agent graph working, one shows code being generated, one shows the Knowledge Graph, one shows a live Camera feed. The office becomes a command center.

7. Comprehensive Physics System

7.1 Core Physics

The physics engine is the muscle system of the organism. It makes the virtual world feel real. All physics are parameterized: users and agents define physics behavior through data profiles, never through code. The engine interprets parameters; no user code ever runs in the physics loop.

Component	Specification

Rigid Body	Mass, center of gravity, moment of inertia. Falling, throwing, stacking, balancing, toppling.
Soft Body	Cloth draping, rope swinging, banner fluttering, deformable objects.
Fluid (simplified)	Particle-based approximation. Water flows downhill, pools form, rain collects.
Constraints/Joints	Hinges (doors), sliders (elevators), springs (trampolines), ball joints (robotic arms).
Custom Gravity	Spherical, directional, zero, variable per zone.
Structural Integrity	Tall structures collapse without support. Weight propagation through stacked objects.
Vehicle Physics	Suspension, tire friction, engine torque, aerodynamics, buoyancy for boats.
Ballistics	Projectile physics for weapons. Bullet drop, wind effect, ricochet off surfaces.
Destruction	Breakable objects with configurable health/damage thresholds. Glass shatters, wood splinters.

7.2 Physics Profile Specification

Every object has a physics profile defined as a data structure (not code). The profile includes: mass, friction, restitution (bounciness), drag, gravity scale, collision shape and dimensions, kinematic flag, constraint definitions, custom force generators (constant, oscillating, attraction, orbit, follow, repel, buoyancy), and trigger definitions (on collision, on velocity threshold, on rest duration).

Agents generate physics profiles from natural language descriptions. Users can manually tweak parameters through the object inspector UI. The engine validates all parameters (no negative mass, no infinite stiffness, bounded frequencies) to prevent physics engine crashes.

7.3 Signal Propagation Physics

A unique feature of Dreamworld: radio waves and infrared signals follow real-ish physics. Players communicate between islands using radio transmitters. The physics engine calculates signal propagation: transmitter power determines range, atmospheric conditions (fog, storms) attenuate signals, mountains and buildings block line-of-sight for infrared. This makes communication itself a gameplay mechanic: you must build and maintain infrastructure, position transmitters strategically, and deal with environmental interference.

Component	Specification
Radio	Omnidirectional. Range based on power. Attenuated by weather. Anyone in range can receive (unless encrypted).
Infrared	Line-of-sight. High bandwidth but blocked by obstacles. Can use reflective relay systems.
Physical Wire	Guaranteed connection between two points. Must be physically laid and maintained. Can be cut by enemies in PvP.

The physics engine doesn't simulate full electromagnetic waves. It uses simplified propagation models: distance attenuation (inverse square law), obstacle occlusion (ray casting), weather modifiers (multiplied attenuation factor). Computationally cheap but produces emergent gameplay.

7.4 Weapon Physics

Guns, melee weapons, and explosives use the physics system for realistic behavior. Bullets are fast projectiles with mass, velocity, drag, and gravity. Melee weapons have swing arcs computed from player animation. Explosives create force fields that push nearby objects and damage those within radius. Weapon configs are physics profiles: a sword has mass, swing speed, damage on collision trigger; an AK-47 has fire rate, bullet mass, muzzle velocity, spread pattern, recoil force applied to the holder.

8. AI Agent System (Nervous System)

8.1 Agent Hierarchy

Every user can have a personal team of AI agents, visualized as an interactive 3D graph in the Agent Manager app on their computer. The hierarchy:

- **Orchestrator Agent:** Receives high-level requests. Decomposes into task graphs. Delegates to supervisors. One per user.
- **Supervisor Agents:** Domain specialists: architecture, code, research, creative, business. Receive decomposed subtasks, further decompose for workers.
- **Worker Agents:** Execute specific tasks. Generate configs, write code, produce content, search knowledge. Many per supervisor.
- **Semantic Agents:** Passive observers. Watch all outputs. Extract meaning. Create Knowledge Graph nodes. Find connections between concepts. Route relevant knowledge to stuck agents.

8.2 Braindump Decomposition

Users speak or type stream-of-consciousness input. The agent decomposes this into structured knowledge: facts extracted, graph nodes created with edges, calendar events detected and scheduled, tasks identified and queued, emotional sentiment tagged, connections to existing graph nodes discovered and linked. The raw conversation is ephemeral; the extracted knowledge is permanent in the local graph.

8.3 Three-Layer Context for Every Response

- **Layer 1 - Personal Graph:** Queried locally on user's device. Semantically relevant nodes selected and sent as context.
- **Layer 2 - Shared Graph:** Platform's shared Knowledge Graph searched for community solutions, patterns, related content.

- **Layer 3 - External Web:** Internet search for current information, documentation, news. Used when personal and shared graphs lack coverage.

8.4 Agent-Generated Content Pipeline

Natural language input triggers the full pipeline: user describes what they want, orchestrator decomposes, workers generate configs (object DNA, physics profiles, interaction scripts, world layouts), the Brain validates configs (checking for impossible physics, broken references, missing exits), valid configs get instantiated in the world. Progressive rendering: partial results appear as each worker finishes so users watch their creation being built piece by piece.

8.5 NPC System

Non-player characters are agents with physical bodies. They have meshes with character controllers driven by AI instead of keyboard input. Features: pathfinding (A* on nav mesh), behavior trees (patrol, idle, talk, fight, flee), dialogue generation (LLM-driven conversations), physics interaction (NPCs can pick up objects, open doors, operate machines). NPCs in story modes, shopkeepers in commercial worlds, robot workers in the greenhouse, enemies in survival mode.

8.6 Telegram Bot Interface

A Telegram bot serves as the mobile-first thin client for the agent system. Users text the bot with requests. The bot wraps messages as AGENT_TASK Packets, routes them to the agent service, and returns AGENT_OUTPUT responses. Users can order code generation, content creation, file management, and world modification from their phone. The bot sends real-time status updates as agents work. Built projects appear on the user's bookshelf when they next enter their 3D house.

9. Knowledge Graph and Memory System

9.1 Three Memory Layers

- **Episodic (Layer 1):** Everything that happens gets logged, tagged, timestamped. Real-time. Object created, task completed, interaction occurred. The raw history.
- **Deep (Layer 2):** Meta-analysis across all episodic data. Extracts patterns: which agent configs work for which problems, user preferences, what 'good' looks like. System intelligence that grows without explicit teaching.
- **Shared (Layer 3):** Community library. All publicly published objects, worlds, designs, solutions, stories. Searchable by everyone. The collective knowledge of the ecosystem.

9.2 Personal Subgraph

Every user has a personal knowledge subgraph stored locally on their device. This graph grows through braindumps, file uploads, agent interactions, and automatic knowledge extraction. It contains:

- **Calendar nodes:** Deadlines, events, class schedules (auto-parsed from uploads).

- **Concept nodes:** Ideas, learning progress, connections between topics.
- **Project nodes:** Active work, code files, world builds, creative outputs.
- **Preference nodes:** Interests, aesthetic preferences, workflow patterns.
- **Social nodes:** Friends, collaborators, group memberships.

The personal graph is visualized as a walkable 3D space. Zoom out to see clusters forming around subjects of deep knowledge. Zoom in to explore individual concept nodes with their connections. Walk through your calendar temporally. The graph IS the user's second brain.

9.3 Upload-to-Graph Pipeline

Any uploaded file (PDF, image, audio, CSV, document) gets processed by a decomposition agent that extracts structured data, creates knowledge nodes, establishes edges to existing nodes based on semantic similarity and temporal proximity, and updates the calendar if time-related data is detected. A class schedule PDF auto-populates the calendar. A lecture PDF generates a concept subgraph with optional learning world generation.

9.4 Learning World Generation

From any uploaded lecture or educational content, agents can generate a learning world: a subgraph of rooms where each room teaches one concept. Rooms contain 3D visualizations of the concept, interactive objects, and a chalkboard question that must be answered correctly to proceed to the next room. Agent-generated questions vary each attempt to prevent brute-forcing. Completion data feeds back into the personal graph (concepts marked as strong or weak). Top of the building offers a bird's-eye view of the full concept map.

9.5 Data Residency Tiers

Component	Specification
Tier 1: Local Only	Personal graph, braindumps, calendar, private notes. Never leaves device. Agent context sent ephemerally, processed in memory, never written to server disk.
Tier 2: Personal Server	Files, projects, code, VPS contents. Synced to user's container for multi-device access. Containerized, isolated, encrypted at rest.
Tier 3: Shared Platform	Published objects, worlds, knowledge nodes. Explicitly shared by user. Stored in platform Knowledge Graph.

9.6 Device Loss and Backup

Encrypted backups of the local personal graph to user's VPS container (Tier 2), user's own cloud storage (Drive, iCloud, Dropbox), or a platform-hosted encrypted blob (decryptable only with user's recovery key/seed phrase). User controls backup destination and frequency.

10. Camera and Recording System

10.1 Recording as Packet Streams

The Camera records game state, not pixels. A recording is a sequence of Packets capturing everything: object positions, physics interactions, player actions, sounds, lighting, weather, camera transforms. Recordings can be replayed from any angle because they contain full world state. Editing after the fact: remove objects, change lighting, slow/speed time, splice recordings. Live streaming is real-time Packet broadcasting.

10.2 Camera Modes

Component	Specification
Handheld	Follows player perspective. Records what you see.
Tripod	Fixed position and angle. Records a specific spot.
Orbit	Circles a target object or player.
Dolly	Moves along a defined path with user-set waypoints.
Drone	Free-flying camera controlled separately from character.
God	Full world state capture without specific viewpoint. For any-angle replay.

10.3 Camera and Watch Integration

Every Camera Packet is timestamped by the Watch. Recordings sync with game time, real time, or music. Timelapse: compress timestamps. Slow motion: stretch timestamps. Synchronized playback with music from the Lungs system.

11. Multiplayer and Networking

11.1 Transport Modes

Component	Specification
Cloud (default)	Player WebSocket connects to Hetzner gateway. Server is authority: resolves conflicts, runs physics, persists state.
LAN	One player hosts locally. Others connect via local IP. Same protocol, same Packets. Sub-1ms latency.
P2P (future)	WebRTC data channels. One player designated host for authoritative physics. NAT traversal required.

11.2 Server Architecture

The backend consists of a Node.js gateway process accepting all WebSocket connections and routing Packets by header, plus dynamically spawned world processes (one per active world,

Node.js), a Python agent service, and a Python knowledge service. Worlds with no players have no running process: zero CPU, zero memory. State sleeps in Postgres until a player enters, at which point the world process spawns, loads from DB into RAM, and begins ticking. On player exit, state flushes to Postgres and the process dies.

11.3 Player State Synchronization

Position updates: 40 bytes per player per tick (position + rotation + velocity + player ID + timestamp). Sent 20 times per second. Interest management: server only broadcasts player positions to nearby players (same world, within render distance). 1000 players online but only 8 on your island means your client receives 8 position updates per tick. Client-side prediction with server reconciliation for responsive feel despite latency.

11.4 Asset Delivery

Large assets (textures, 3D models, audio) delivered over HTTP/CDN, not WebSocket. Browser cache stores assets with Cache-Control headers. Version numbers on object configs trigger re-fetch on change. WebSocket carries only small real-time Packets. Initial world entry: one bulk state Packet loads all objects; subsequent changes sent as deltas only.

11.5 Offline and Reconnection

Grace period: server keeps player session alive for 30-60 seconds on disconnect. Character stands still. On reconnect, client reconciles missed state. Offline sandbox mode: build locally, sync to server when back online. Conflict resolution for offline edits: last-write-wins for personal content, merge for collaborative content.

12. Gameplay Modes

12.1 Sandbox (Default)

Free creation mode. Build anything, explore anywhere, create worlds, generate objects, upload content, customize your house, code programs, train agents. No objectives. No constraints. The default experience for every user.

12.2 Story / Survival Multiplayer

The flagship story mode is a multiplayer survival game set in the floating cloud island graph. The narrative: your island's infrastructure is damaged. You must:

1. Repair your power source to bring your computer and communications online.
2. Build and maintain infrastructure: radio transmitters, solar panels, water collection, food production (AI farming).
3. Establish communication with other islands via radio and infrared links.
4. Explore the constellation: fly or travel to other islands for resources, allies, trade.

5. Defend your base: build defenses, program robot guards, arm yourself with weapons.
6. Survive threats: hostile NPCs, environmental hazards, PvP raiders in contested zones.
7. Progress through the tech tree: better computers, faster planes, stronger weapons, smarter AI.

The survival graph contains thousands of dormant nodes that activate when players approach. Each node has procedurally generated or agent-generated content. The graph is massive enough that full exploration takes months, and community efforts map it collectively through the shared Knowledge Graph.

12.3 PvP Free-For-All

Designated danger zones in the graph. No item protection: anything you bring in can be stolen. Free-for-all combat with weapon physics. Loot drops from defeated players. High risk, high reward: rare resources and powerful items spawn only in PvP zones. Timed events: king-of-the-hill, team battles, drone racing, vehicle combat.

12.4 User-Created Stories

Any user can create story experiences as subgraphs with scripted events. Event portals placed in the world lead to story subgraphs. Stories are built with trigger chains: player enters room, trigger fires, NPC appears, dialogue plays, choice presented, outcome determines next trigger. Agents assist with story creation. All user stories are free to create and play. Shared in the Knowledge Graph. Rated by the community.

12.5 AI Farming

Players can establish automated farms on their islands. Crop growth follows the Watch system (real-time growth cycles). AI agents optimize planting, watering, and harvesting schedules. Drone surveillance monitors crop health. Robots (programmable NPC objects with physics) perform physical farming tasks. Farm output feeds into survival resources or marketplace commerce. The greenhouse from the organism architecture is realized here as a core gameplay mechanic.

13. Economy and Commerce

13.1 Token Economy

Tokens are the platform currency, pegged to real compute costs. Token pricing is transparent: each action's token cost correlates directly to the server resources consumed.

Component	Specification
Small world creation	Free (minimal compute/storage)
Large world with complex physics	Tokens (more Brain/Muscle compute)
AI agent object generation	Tokens (LLM API calls)

Knowledge Graph storage	Tokens proportional to size
VPS computer upgrades	Tokens per month based on tier
Premium features	Tokens (advanced physics, priority agents, larger worlds)

13.2 The Shop

Roan (the platform creator) operates the official shop, selling curated objects, character templates, world templates, themes, and premium items. The shop is a physical world in the constellation that players can visit. Only invited creators can sell on the platform to maintain quality control and prevent scams. Application process for potential sellers. All marketplace items are reviewed before listing.

13.3 Curated Marketplace

Unlike open marketplaces that get flooded with low-quality content, Dreamworld's marketplace is invite-only for sellers. This maintains the platform's soul and aesthetic quality. Sellers apply, demonstrate their work, and receive approval before gaining marketplace access. Platform takes a percentage of each sale. Buyers have quality assurance because everything is curated.

13.4 Business Storefronts

Businesses that want a presence in Dreamworld pay for a storefront world. They can display products (iframe their existing web shop onto screens), host events, create branded experiences. Monthly fee plus transaction percentage. Businesses add value (content, variety, commerce) while funding the platform.

13.5 Revenue Streams

- **Token purchases:** Direct compute/storage credits.
- **Computer upgrades:** Monthly VPS tier fees.
- **Marketplace fees:** Percentage cut of all sales.
- **Business storefronts:** Monthly presence fee + transaction percentage.
- **Enterprise licenses:** Schools, companies wanting private instances.
- **Creator program:** Invited sellers share revenue with platform.

14. Communication Systems

14.1 In-Game Communication

- **Text chat:** EVENT Packets with schema chat_message. Local (nearby players), world-wide, or direct messages.
- **Radio system:** In-game radio transmitters with physics-based propagation. Broadcast on frequencies. Others tune in. Range limited by transmitter power and weather.

- **Infrared:** Line-of-sight data links between islands. High bandwidth for file transfer. Blocked by obstacles and weather. Relay systems with mirrors.
- **Physical mail:** Send physical letter objects through the postal system between islands.
- **Phone objects:** Craftable phone item for player-to-player voice/text (future: WebRTC).
- **Proximity voice (future):** WebRTC spatial audio. Hear players near you in 3D space with volume based on distance.

14.2 Working Phones and Computers

Phone objects are inventory items with a small screen interface (same screen-object architecture). Text messaging between players, contact lists, notification feed. Computers are the full-featured version with OS, apps, and VPS backing. Both use the same Packet system for all communication.

15. Safety, Moderation, and Legal

15.1 Content Safety Architecture

Layered protection system (the Immune System):

- **No image uploads at launch.** All visual content comes from curated library, templates, or agent generation pipeline (which passes through classification before rendering).
- **Agent output validation:** All agent-generated content passes through a classifier before world instantiation. Flagged content is quarantined.
- **Physics validation:** Config validator checks for impossible values, broken references, missing exits, degenerate geometry before instantiation.
- **Griefing prevention:** Unauthorized deletion blocked in non-PvP worlds. Object spam detection and throttling. Rate limiting on creation.
- **Malicious code sandbox:** All user scripts (interaction scripts, custom programs) run in sandboxed containers with no access to other users' data, no unauthorized network access, strict resource limits.
- **Knowledge Graph quality:** Semantic analysis detects low-quality nodes. Community ratings feed in. Repeated low-quality contributions trigger review.

15.2 Reporting System

Every object, world, and player has a report button. Reports create MODERATION Packets containing: reporter ID, target ID, category (harassment, explicit content, griefing, spam, other), and optional description. Reports flow to a moderation dashboard. Auto-escalation: 3+ reports from different users within 24 hours auto-hides content pending review. False reporters tracked and warned.

15.3 Human Moderation

Automated systems catch obvious violations. Human moderators handle nuance: harassment, offensive-but-technically-legal builds, thinly veiled threats, contextual abuse. Community moderator program as platform grows. Clear appeals process. Transparent policies.

15.4 Early Beta Safety Warning

Displayed prominently during onboarding: 'This platform is in early beta. Do not store passwords, financial information, or sensitive personal data. Treat this like a public notebook. We are actively building security infrastructure and will update this guidance as the platform matures.'

15.5 Security Minimums at Launch

- **Passwords:** Hashed with bcrypt or argon2. Never plaintext.
- **Transport:** All connections over HTTPS/WSS. TLS certificates on all endpoints.
- **Database:** Postgres requires authentication. Not exposed to public internet. Only backend processes connect.
- **Agent service:** Ephemeral processing. No personal context written to server disk. Containers reset after requests.

15.6 Legal Requirements

- **Terms of Service:** User agreement covering acceptable use, account termination, content ownership, liability limitations.
- **Privacy Policy:** What data is collected, how stored, who has access, how to delete. GDPR-compliant for European users.
- **DMCA Process:** Contact email for copyright takedown notices. Timely response. Counter-notice process.
- **Age Restriction:** 13+ requirement in ToS. COPPA compliance if targeting schools with younger students.
- **Content Licensing:** Publishing flow requires license selection: public domain, attribution required, non-commercial, all rights reserved. Fork lineage tracked in Knowledge Graph.

16. Visual Programming System (The Eyes)

Code rendered as physical, interactive, walkable 3D space with real physics. Every programming construct has a predetermined 3D form based on semantic meaning.

16.1 Semantic Mapping

Component	Specification
for loop	Spiral/ring. Rotation speed = iteration speed. Data elements orbit inside.
while loop	Treadmill/conveyor. Runs until condition flag drops.

if/else	Fork in path with physical gate/switch.
function	Portal/doorway. Walk through to enter scope.
variable	Container/box. Weight proportional to data size. Glows when accessed.
array	Shelf/rack. Items in indexed slots.
return	Output pipe. Objects travel through and exit.
class	Building. Rooms = methods. Furniture = properties.
recursion	Spiral staircase descending into itself.
try/catch	Safety net below tightrope.
async/await	Conveyor with pause button.
API call	Messenger pigeon flies off, returns with package.

16.2 Fractal Zoom Levels

- **Level 1 - Continent:** Whole program as landscape. Mountains = heavy modules. Rivers = data flows.
- **Level 2 - City:** Module/class. Buildings = classes. Streets = method calls.
- **Level 3 - Building:** Function. Floors = block scopes. Rooms = control structures.
- **Level 4 - Room:** Control structure. Data on conveyors. Comparison gates.
- **Level 5 - Furniture:** Individual operation. Pipes connecting containers. Binary bits visible.

16.3 Interactive Manipulation

All code visualizations have real physics. Users can: grab variable containers and carry them to different scopes (reassignment), spin loop spirals faster (modify bounds), flip if/else switches (debug override), disconnect pipes (fault injection), build new structures by placing primitives (visual coding). Multiple people can walk through the same code simultaneously for collaborative debugging.

16.4 Custom Themes

Visual programming themes are customizable via config: factory theme (loops = conveyor belts), organic theme (loops = heartbeats), medieval theme (loops = watermills), space theme (loops = orbital paths). Community creates and shares themes through the Knowledge Graph.

17. The Creative Breathing Cycle (The Lungs)

A non-negotiable daily practice woven into the platform. Every other day: make music (inhale, 30 minutes). Alternate days: make art while playing the music, recording the session as a music video (exhale, 30 minutes). Creative output feeds the ecosystem:

- **Music:** Uploaded to in-game iPod. Assignable to objects (music box plays your track). Playable on radio. Ambient soundtracks for worlds.
- **Art:** Displayed on museum canvases. Mappable as textures on any surface. Used as world decoration.
- **Recordings:** Camera captures art sessions. Become content in the Knowledge Graph. Shareable as experiences.

18. Sound System

Spatial audio using Web Audio API. Sounds have 3D position, direction, distance attenuation, and environmental effects (reverb in caves, muffled through walls).

- **Ambient per world:** Wind, water, birds, machinery. Defined in world config.
- **Object sounds:** Music boxes, waterfalls, engines. Attached to objects.
- **Physics sounds:** Collision = impact, breaking = shatter, footsteps vary by surface material.
- **UI sounds:** Computer activation hum, keyboard clicks, button feedback.
- **Music playback:** iPod player, radio broadcasts, concert hall venues.
- **Communication:** Radio static, infrared data chirps. Functional sounds that convey system state.

19. Time System (The Watch)

Component	Specification
Real time	Synced to UTC. Converted to user's timezone for display. Drives calendar.
Game time	Per-world configurable. 1:1 real, 10-minute days, eternal twilight.
Day/night cycles	Sky rotation, lighting shifts, stars appear, moonlight. Per-world config.
Event timing	Scheduled triggers: crop growth, watering, agent timeouts, PvP events.
Session tracking	Time online, time per world, time since login. Feeds Deep Memory.
Version history	Timestamps everything. Enables world rollback to previous states.
Metronome	Creative pulse. Syncs music playback. Controls animation timing.

All timestamps stored as UTC internally. Converted to user's current timezone for display. Time zone edge cases (travel, DST changes) handled by always computing from UTC.

20. Authentication and Identity

20.1 Account System

Email/password registration with bcrypt hashing. Future: OAuth providers (Google, GitHub, Discord). One identity across the entire platform: one avatar, one inventory, one personal graph, one reputation. Session tokens (JWT) over HTTPS. WebSocket connections authenticated on upgrade handshake.

20.2 Social Graph

- **Friends list:** Mutual connections. See online status, current world.
- **Reputation:** Community rating based on contributions, reports, creation quality.
- **Groups/Factions:** Player-created organizations. Shared worlds, communication channels, collaborative projects.
- **Blocking:** Per-user blocking prevents all communication and visibility.

21. Technical Infrastructure

21.1 Deployment Architecture

Component	Specification
Server	Hetzner dedicated/cloud. Single machine to start, horizontal scaling later.
Gateway	Node.js process. Accepts WebSockets. Routes Packets by header.
World Processes	Node.js child processes. One per active world. Spawned/killed dynamically.
Agent Service	Python process. Handles all LLM API calls and agent orchestration.
Knowledge Service	Python process. Manages Knowledge Graph, search, semantic analysis.
Database	PostgreSQL. Spatial indexing (PostGIS) for proximity queries.
Asset Storage	Local filesystem or Hetzner object storage. HTTP delivery with cache headers.
Containers	Docker for service isolation. User VPS containers for personal compute.

21.2 Performance Budgets

- **Physics tick:** 50ms budget at 20 ticks/second. Max ~500 physics objects per world in Node.js.
- **WebSocket:** Target <100ms round-trip for position updates on cloud.
- **World load:** Target <2 seconds from entering a world to fully rendered.

- **Agent response:** Target <15 seconds for simple generation, progressive rendering for complex tasks.

21.3 Scaling Strategy

Phase 1: Single Hetzner box runs everything. Phase 2: Separate machines for gateway, world processes, agent service, database. Phase 3: World processes distributed across multiple machines with gateway routing to correct host. Phase 4: CDN for static assets, read replicas for database, regional gateway nodes for global latency reduction.

21.4 Monitoring and Recovery

Process health monitoring with automatic restart on crash. Database backups (daily full, hourly incremental) to separate storage. Alerts on: world process crash, tick rate drop below 15fps, database latency above 100ms, agent service unresponsive, disk usage above 80%. All logs centralized and searchable.

21.5 Future Renderer Consideration

Three.js with WebGL for initial launch. WebGPU renderer path planned for next-generation performance. Renderer abstraction layer prevents coupling to WebGL-specific features. WebGPU provides: significantly faster rendering, compute shaders for client-side physics, better memory management.

22. Data Ownership and Portability

Users own their creations. The platform provides full data export:

- **World export:** Full world config (JSON) plus all referenced assets downloadable as a package.
- **Object export:** Individual object configs with physics profiles, interaction scripts, textures.
- **Personal graph export:** Full knowledge graph in standard format (JSON-LD or similar).
- **Code and files:** VPS container contents downloadable as archive.
- **Account deletion:** Complete removal of all platform-stored data. Local data persists on user's device.

Open-source engine means users can self-host with their exported data. The platform is confident enough in its value (accumulated Knowledge Graph, community, network effects) that data portability strengthens rather than weakens the proposition.

23. Release Plan

23.1 Pre-Alpha: Telegram Bot (Weeks 1-4)

Ship the Nervous System standalone. Telegram bot that accepts natural language, routes to agent service, returns generated code/content. Add user accounts, token tracking, basic Stripe payment. This validates the agent architecture and generates early revenue.

23.2 Alpha: The House (Months 1-3)

Build the 3-month polished MVP. One world, one house, one office, one computer, one agent. Every detail agonized over: lighting changes with time of day, camera transitions feel like breathing, the agent responds with awareness and personality, the bookshelf displays knowledge graph nodes as physical books. No multiplayer. No portals. Just the single-player experience polished to the point where people forget they're in a browser.

23.3 Beta: The Constellation (Months 4-8)

Multiplayer goes live. Multiple worlds in the graph. Portals between worlds. Flying between islands. Template worlds for new users. Character customization. The map. Routes. LAN mode. PvP zones. Basic survival mechanics. Object creation through agents. The shop opens with curated items.

23.4 Launch: The 3D Internet (Months 9-12)

Full feature set. User-created stories. Business storefronts. The visual programming system. Learning world generation. Drones. Vehicles. Signal propagation physics. Comprehensive weapon and combat system. Enterprise licensing. Mobile app. Community moderation program. The Knowledge Graph is rich with community content. The organism breathes on its own.

23.5 Seeding: Pre-Launch Content

Before any public release, the platform must feel alive. Manually and agent-assisted creation of: 50+ interesting worlds, a library of objects, several playable stories, a populated Knowledge Graph with useful information, template characters, preset physics configurations. First users must feel they are joining something living, not staring at empty space.

24. Feature Priority Matrix

P0 = Blocks everything. P1 = Required for alpha. P2 = Required for beta. P3 = Required for launch.

Feature	Priority	Phase	Complexity
Packet protocol specification	P0	Pre-alpha	Medium
PostgreSQL schema design	P0	Pre-alpha	Medium
Auth (email/password + JWT)	P0	Pre-alpha	Low
WebSocket gateway	P0	Pre-alpha	Medium

Character controller (kinematic)	P0	Alpha	High
Camera spine (scroll zoom)	P0	Alpha	Medium
Template house + office	P0	Alpha	Medium
In-game computer (HTML overlay)	P0	Alpha	Medium
Agent service (Python + LLM)	P0	Alpha	Medium
Braindump decomposition	P0	Alpha	High
Personal knowledge graph (local)	P0	Alpha	High
Telegram bot	P0	Pre-alpha	Low
Lighting and atmosphere	P1	Alpha	Medium
Sound system (spatial audio)	P1	Alpha	Medium
Bookshelf / file system	P1	Alpha	Low
Calendar integration	P1	Alpha	Medium
World process spawning	P1	Beta	High
Multiplayer sync	P1	Beta	High
Character customization	P1	Beta	Medium
Tattoo system	P1	Beta	Medium
Flying / bird view	P1	Beta	Medium
Portals between worlds	P1	Beta	Medium
Physics engine (rigid body)	P1	Beta	High
Object creation via agents	P1	Beta	High
Preset object library	P1	Beta	Medium
Map / route planning	P1	Beta	Medium
LAN multiplayer	P2	Beta	Low
PvP free-for-all zones	P2	Beta	Medium
Weapon physics (guns, melee)	P2	Beta	High
Vehicle physics (cars, planes)	P2	Beta	High
Drone system	P2	Beta	Medium
Signal propagation (radio/IR)	P2	Beta	High
Survival mechanics	P2	Beta	High
AI farming	P2	Beta	Medium
Robot NPCs	P2	Beta	High
Shop / marketplace	P2	Beta	Medium
Token economy + Stripe	P2	Beta	Medium
Reporting system	P2	Beta	Low

Monitor + Computer + Wire objects	P2	Beta	Medium
VPS containers per user	P2	Beta	High
Story creation tools	P3	Launch	High
Visual programming system	P3	Launch	Very High
Learning world generation	P3	Launch	High
Soft body / fluid physics	P3	Launch	Very High
Camera recording system	P3	Launch	Medium
Business storefronts	P3	Launch	Medium
Enterprise licensing	P3	Launch	Medium
Voice chat (WebRTC)	P3	Launch	High
Working phones (inventory item)	P3	Launch	Medium
Image upload with AI moderation	P3	Launch	High
World versioning / rollback	P3	Launch	High
Custom visual programming themes	P3	Launch	Medium
Deep Memory meta-analysis	P3	Launch	Very High
Mobile native app	P3	Post-launch	Very High
WebGPU renderer	P3	Post-launch	High
P2P multiplayer (WebRTC)	P3	Post-launch	High

25. The Soul

Wake up at the same time daily. Attend class. Work out consistently. 15 minutes of planning to start the day. 15 minutes of reflection to end it. Get real plants for the house. Lock the sleep schedule. Bolivia by end of year. Keep searching for what God is. Make robots. Make music and art every other day, 30 minutes, non-negotiable. Stop pulling triple all-nighters.

Remember: the organism is a tool, not a replacement for living. Real world still exists. Real plants, real music, real silence. The 3D internet is powerful because it extends reality, not because it replaces it.

Build the first bone. The rest of the body will grow around it.

E N D O F D O C U M E N T